

EMAIL AUTOMATION USING LARGE LANGUAGE MODELS

1st Vivek Kumar
Department of CSE
GNIOT
Greater Noida , India
vivekshivay6@gmail.com

2nd Vatsal Chaudhary
Department of CSE
GNIOT
Greater Noida , India
vatsalchaudhary01@gmail.com

3rd Shreyansh Singh Rathore
Department of CSE
GNIOT
Greater Noida , India
rishus9151@gmail.com

4th Vishveshwar
Department of CSE
GNIOT
Greater Noida , India
vishusindhav555@gmail.com

5th Arjun Singh
Department of CSE
GNIOT
Greater Noida, India
innovativearjunsigh@gmail.com

6th Dr. Asif Khan
Professor and dy.HOD CSE
GNIOT
Greater Noida, India
asif@gnit.net

Abstract

Although email remains the primary vehicle for business communication, writing emails by hand is typically slow, frequently inconsistent with their intended tone, and generally full of mistakes. This document describes how we built a practical email automation system based upon large language models (LLM), which consists of taking a small number of user's inputs (i.e., recipient, subject, tone, and important points) and returning a complete email draft that is ready to send by using an API call to a hosted LLM. The email system consists of four separate layers: an interface layer for the client to enter information about their email, an email generation function layer where the letter is assembled and stored for creating future letters, a database of previously generated letters, and finally an LLM to generate new letters. We compared the completed email drafts produced by our system with emails produced using traditional drafting methods, template generator, and AI Mail Assistant for a total of twenty usage scenarios and fifteen participants. The results clearly indicate that the email system reduced average email drafting time by approximately 70 %, minimized editing time for the email drafts to less than 15 %, and achieved over 4.5/5 satisfaction rating from users. System architecture has been designed to allow future expansion to include support for multiple languages, tone adjustment based on sentiment analysis, and on-device processing for sensitive emails.

Index Terms:

email automation, large language models, natural language processing, generative AI, intelligent communication systems

1.INTRODUCTION

Email is still the backbone of formal digital communication in business, academia, and government. Even with the rise of instant messaging apps and collaboration platforms, email remains the main channel for contracts, official notices, and stakeholder updates [1]. According to a McKinsey Global Institute study, the average knowledge worker spends close to 28 % of the working week on email—writing, reading, and managing replies [2]. That is a large chunk of time, and it points to a real need for smarter tools that can take over the repetitive parts of email drafting. The issue is much larger than simply the amount of time they are wasting. Employees will frequently have difficulties maintaining a consistent tone while working on

different types of messages and also will often make grammatical mistakes (or awkward phrase choices) when under time constraints. Studies conducted regarding workplace communications demonstrate that an email written poorly can damage one's credibility, result in confusion related to business processes, and potentially lead to conflict with the reader misinterpreting the sense of tone intended by the writer. Therefore, the real rationale for automating this type of process includes both saving time and improving the quality of the outgoing emails. With traditional email assist features, most have relied upon using fixed templates or basic rule based suggestion engines [3]. These work fine for routine messages like order confirmations, but they fall short when the situation calls for nuance—say, a diplomatically worded complaint or a warm follow-up to a client meeting. The emergence of large language models, starting with the GPT family, has changed the game by making it possible to generate coherent, context-aware text from short prompts [4][5]. In this paper we describe an end-to-end email automation system that puts these LLM capabilities to practical use. Users provide input in four different ways: they complete five fields: (1) Choose a recipient, (2) Identify the subject line, (3) Select the type of tone desired, and (4) Provide the important items to remember. A structured prompt will be created and sent to an e-mail -generating program which includes an email-generating machine language model. Users then have the option of previewing the finished email to edit it prior to transmission via the internet. This gives the user the opportunity to maintain a connection to the mail process while reducing the amount of time required for drafting a new message significantly through automation.

There are three important things to note about our contributions. The first is the development and implementation of a modular, technology-independent pipeline for the generation of e-mails that separates into four distinct components the user interface (publicly), the processing logic from the user interface, the storage of data, and the inference from the machine model used for generating e-mails. Second, we run a controlled experiment comparing our system against manual writing, template tools, and native inbox AI assistants on five quantitative metrics. Third, we map out a concrete roadmap for future extensions—multilingual output, sentimentadaptive tone, and on device inference for situations where data privacy is paramount. From an organizational standpoint, even a modest per-email time saving adds up fast when you consider that a mid-sized

company might send tens of thousands of outbound messages each day. Consistent tone and fewer grammatical mistakes also reduce the reputational risk that comes with sloppy or misinterpreted emails—a concern that matters most in client-facing and regulated communication. This paper has the following organization. The literature on email automation will be reviewed in Section II beginning with rule-based template systems, continuing to statistical language modeling systems, and concluding with modern-day neural network types of systems. The system architecture will be described in section III (including a detailed discussion of each component). Key implementation decisions (e.g., prompt development and error management) will be discussed in section IV; section V will describe the experimental setup and results; section VI will conclude; and, finally, section VII will describe future work.

2. LITERATURE REVIEW

Work on email assistance tools can be grouped into three broad waves. The earliest systems used hardcoded templates and keyword-triggered rules to fill in predefined message skeletons with user-supplied details [3]. They were cheap to run but almost completely rigid: if the message did not fit one of the templates, the output was unusable. Their sweet spot was narrow—order confirmations, appointment reminders, and other messages where variety is minimal and a stiff tone does little harm.

The following methods were statistical. Statistical techniques for creating basic next-word prediction and spam filtering by implementing n-gram models and naive Bayes classifiers were developed, which ultimately had a significant impact on improving user experience with email while not directly benefiting users via improved quality of their composition - with the launch of Google's Smart Compose feature in 2018. This tool utilizes Recurrent Neural Networks trained on anonymized email data and provides users with inline suggestions for how to complete incomplete sentences. However, it does not allow for the generation of entire messages and does not allow for adjustment of tone, thus making it less useful in situations where users wish to control the tone of their communications.

The introduction of the transformer architecture developed by Vaswani et al. marked a dramatic change in the use of attention-based architectures, which significantly increased both the training speed of these models, as well as the coherence of text produced by these models. The successors of the transformer architecture (GPT-2, GPT-3 and subsequent instruction-tuned models) provided clear and compelling evidence, that if sufficiently large, the resulting language models can produce extended, contextually appropriate texts across multiple domains, with little to no need for domain-specific fine-tuning. In the case of email communications, the implementation of self-attention as developed in the transformer model allows for the retention of consistency across all components of a multipara graph email, (e.g. salutation, body and closing keying). Additionally, recent commercial products (e.g. Microsoft's Copilot) use variants of the transformer-based architectures to produce the same type of text generation capabilities described above.

Sahoo et al (2014) created an open source tool that sections out various ways you can design an interface for a Large Language Model: 1) Zero-shot; 2) Few-shot; 3) Chain-of-Thought. Their paper cataloged both strategies and evaluated how to prompt the model to produce text in a certain form based upon an example. Therefore, based on the findings of Sahoo and team, Gehrmann et al (2014) reviewed evaluation methods for LLM generated text, identifying grammar, coherence, and adequacy as primary criteria for the quality of machine. The work we present here adds to this body of literature by creating a fully functional deliverable system (the tool mentioned earlier) that allows users to control the tone of their LLM through prompt engineering as well as evaluating quality in all three categories while measuring efficiency. Additionally, gaps exist within the academic literature regarding the lack of a unifying, platform independent, mechanism which integrates either A) full draft generation; B) the ability to explicitly choose tone; and C) transparent prompt engineering. The commercial product provides 'a polished interface' to the end user, however, the proprietary nature of many commercial products inhibits investigators from examining the underlying foundations of the product with regard to LLM prompting strategies in their entirety. The work we outline will provide a systematic and documentable method for developing an efficient modular pipeline that can be used as both a practical tool and a data collection device to explore the effectiveness of LLMS in computer aided communication; specifically email automation. Due to the fragmented nature of email generation research conducted thus far in academia on LLM's there remains a dearth of relevant literature.

3. SYSTEM ARCHITECTURE

A. Architectural Overview

The architecture of our solution is structured into four distinct layers: presentation, processing, persistence, and inference. Each of these layers can be developed independently and the swapping out of a component (i.e., switching from one provider of LLM to another) will not disrupt the functionality of the other layers. The design of this layered system incorporates standard principles from the Service-Oriented Architecture™ domain; thus, each layer has no visibility into the inner workings of its adjacent layers but rather has clearly defined interfaces to other layers.

An overview of how user input flows through the entire system is illustrated in Figure 1. User input comes into the presentation layer; it passes through both the processing and prompt generation, and finally, it goes to the LLM for synthesis of a draft. The final draft goes back to the interface for review and optional modification, prior to emailing the final variant to the user. The process described above maintains total user control of the automated text being sent under the individual's name in a business context.

B. Presentation Layer

A display layer or presentation layer provides the user with a web-based interface that enables him/her to enter the recipient address, subject line, tone (formal, semi-formal, casual or urgent), and main content (free text) for the email before it is processed. Once submitted, the presentation layer sends a structured JSON message (payload) over a RESTful API to the

processing layer. An editable preview of the email will then be displayed in the presentation layer for the user to review and edit before sending. The layout of the tool will adjust to accommodate other screen sizes so that it can be used successfully by either a desktop computer or a mobile device.

C. Processing Layer

The purpose of the processing layer is to be the orchestration hub for all generations of new contents. The processing layer verifies all inputs when a generation request is received; it will then create a composite prompt that has both tone directives and content specifications and send that to the Inference Layer. The processing layer also handles user sessions, creates rate limits to stop abuse, and records metadata about each generation request for future analysis. All communication between Layers is encrypted. There is a request queue within the processing layer that will handle any traffic spikes to ensure that concurrent users will not degrade the quality of the response or run through the API's rate limits.

D. Persistence Layer

The persistence layer is a relational database that stores user profiles, email generation history, and system configuration. The schema has been normalized to a third normal form to prevent the duplication of data, and it contains indexes for commonly queried fields such as user ID, timestamp, and email status. Keeping track of email generation history allows for the development of additional features in the future such as personalized style learning and use dashboards. Organizations can use retention policies to ensure compliance with datagovernance policies by automatically deleting records when they reach a certain age.

E. The Inference Layer

communicates with a hosted large language model using a standard API. The user's inputs are wrapped in a structured prompt that contains a system-level instruction for how the model should behave including tone of voice, length of material created and any formatting rules. The LLM takes this prompt, generates a complete email draft and sends the draft back to the processing layer to be returned to the front end of the application. This layer has been designed with abstraction of the provider in mind, so that changing from one LLM provider to another can be done via configuration changes vs. code changes.

F. Inter-Layer Communication and Security

Layered communication across layers has a request/response style over REST endpoints. Each endpoint strictly validates its input and rejects messages that are malformed prior to hitting the core logic. All email is secured in transit using TLS to ensure email remains encrypted once sent. When a user logs in, the user is issued an authentication token which must be included with requests; authentication tokens automatically expire and renew so that stale credentials are blocked from accessing the service.

4. IMPLEMENTATION

A. System Development

The four subsystems of the prototype work together. The front end, which is a single page built using a reactive component-based UI framework and has reactive form fields plus a draft preview that updates in real-time, does not block the user's ability to interact with the UI while the backend builds the user's

model via asynchronous (promise-based) HTTP Ajax requests. Server-side processing is done with a modern web-based "framework" that has exposed standard RESTful/API endpoints for user management (account), email generation and history. Each of the services created by dependency injection are separate entities therefore all three (prompt construction, API communication and data access) are independent components. The validation of input occurs at the controller level using declarative constraints, whilst other issues that are not specific to this application: logging and authentication, are handled by middleware.

B. Prompt Engineering

The prompt design of how to implement an application will have an impact on the system's overall performance. The prompt for the applications will consist of three components: (1) a system-level directive to let the language model know that it is a professional email author, (2) a tone-based directive, based on the user selection, and (3) a user-level directive that contains the recipient information, subject line, and key points. The three-part structure of the connection between the user selection and the applications will create consistency across each of the email types yet still allow for differences in tone and content. Through iterative refinements, we developed multiple versions of each application by testing each version against a set of predefined email scenarios to ensure that any planned modification to the output would not negatively affect the previously successful output.

C. Data Persistence

The persistence subsystem enables a mapping of objects to relations between developer-created entity objects (rather than using raw SQL) by using a layer of object-relational mapping. The definitions of the entities are analogous to that of the normalized schema, and cascading rules provide referential integrity between related tables. Every database migration is subject to a version control process to help ensure that the schema can be replicated across different environments.

D. LLM Service Integration

The service class dedicated to working with the language model includes all interaction with the model (i.e., API authentication, request serialization, response parsing, and error handling). A retry mechanism is built into this class (with an exponential backoff for transient API failures) to allow API calls that experience some form of interruption to be retried and get through those interruptions. Configurable timeouts help keep the service's request to send through to the language from waiting too long in the event of slow network connectivity.

Overall, security was considered at every level of development. User-provided passwords are hashed using an industry-standard algorithm along with a unique salt for each end-user. API keys required to use the language service are kept in environment-specific configuration files and not directly in the source code, so there would be no potential leak via version control. Input sanitization occurs prior to any data supplied by the end-user being created into a prompt to help protect against prompt injection attacks which could alter the behavior of the model.

Each request generated for the language model will have a unique trace ID that travels through all of the processing steps, making identifying problems an easy task. The major performance metrics of request volume, percentage of time to return a response and error percentage are provided through a monitoring endpoint, providing operational visibility to the service in real time.

The testing strategy consists of three tiers. Unit tests cover the individual services (i.e. methods that make up the service), including prompt formatting and input validation. Integration tests cover the entire process of executing a request, from submitting the request via the API, to how that request interacts with the database. End-to-end tests cover the user experience in the service through the front and back end of the system.

5. RESULTS AND DISCUSSION

A. Experimental Setup

We conducted an experiment in which there were 20 different email scenarios researched, broken down into five different categories (formal business email, informal internal email, customer email, meeting email, complaint or escalation email). Each scenario had a defined recipient context (who it is going to), subject matter (what the email is about), and tone (how the email should read). For each scenario, we tested four email composition methods: (1) completely from scratch, (2) a traditional mail-merge template tool, (3) built-in AI (Gmail Smart Compose/Outlook Copilot), and (4) our proposed LLM-based system.

Fifteen people with different levels of experience writing professionally were recruited to compose the emails using each of the four methods. By taking five different measurements during each trial: (1) total drafting time (in seconds from beginning through to submit), (2) total percentage of tokens that were edited or deleted from the generated composition prior to sending, (3) total grammar score as assessed by Grammarly's automated grammar scoring system on a score from 0-100, (4) level of user satisfaction based on a five-point Likert Style Scale, and (5) the amount of time the user had to wait after requesting a draft until they received it (in milliseconds) for the automated methods only. All participants provided informed consent for their participation and the study protocol was approved by the institutional project committee.

B. Comparative Feature Analysis

Table I compares the four methods across eight functional dimensions.

TABLE I: FEATURE COMPARISON ACROSS EMAIL COMPOSITION METHODS

Feature	Manual	Template	Inbox AI	Proposed
Full draft generation	Yes	Partial	Partial	Yes
Tone customization	Manual	No	Limited	Yes
Context awareness	High	None	Moderate	High
Grammar correction	No	No	Yes	Yes
Drafting speed	Slow	Fast	Moderate	Fast
Personalization	High	Low	Moderate	Moderate
Platform independence	Yes	Varies	No	Yes
Learning curve	None	Low	Low	Low

Only our solution allows you to create drafts automatically, choose a tone and remain neutral and standard regardless of where they are eventually published or sent from. When drafting by hand, you are able to create the most personal and contextually appropriate draft; however, it takes much longer than if you were drafting with our system and also does not

provide you with as much consistency with grammar, especially if you are rushing to meet a deadline.

C. Drafting Time Comparison

TABLE II: AVERAGE DRAFTING TIME (SECONDS) PER EMAIL CATEGORY

Category	Manual	Template	Inbox AI	Proposed
Formal business	342	145	186	98
Informal internal	218	102	124	64
Customer service	396	168	210	112
Meeting coordination	264	120	156	78
Escalation / complaint	410	192	238	126

For every category, it took an average of 95.6 seconds to generate each email using the new system. Compared to manual email generation (average of 326.0 seconds), this represents a 70.6% decrease. Compared to template-based email generation (average of 145.4 seconds), this represents a 34.2% decrease. Compared to native AI email generation in an inbox (average of 182.8 seconds), this represents a 47.6% decrease. The category that showed the greatest improvement was the escalation and complaint category since careful tone management is crucial; in addition, manual generation took the longest time to complete.

Using paired t-tests, we found that there are statistically significant differences ($p < 0.01$) between the email creation times of our solution compared to each of the other three competing solutions for all five categories of emails. The consistent magnitude of the improvements across the five categories indicates the new system is expected to yield similar benefits regardless of the type of email being sent.

D. Editing Effort Analysis

TABLE III: POST-GENERATION EDITING EFFORT (% TOKENS MODIFIED)

Category	Template	Inbox AI	Proposed
Formal business	38	22	14
Informal internal	32	18	10
Customer service	42	26	16
Meeting coordination	28	16	8
Escalation / complaint	48	30	18

The average percentage of tokens that required editing by participants (14.0%) was lower for our output than for either the template output (38.0%) or the inbox drafts produced by AI (22.0%). The low edit overhead for our output shows that the three-part prompt structure produced content closely matched to the user's desired meaning and tone. More specifically, the majority of edits made to our output were cosmetic in nature, such as slight rewordings of phrases or changes of proper nouns that the model could not have known. In contrast, most edits made to template-generated output involved restructuring whole paragraphs and entire sections, indicating that rigid templates do not provide the necessary flexibility to address the many different types of situations encountered in sending emails in real life.

E. Grammar Quality and User Satisfaction

The accurate grammar produced by our product was confirmed on average at 95.2/100. The scores show that this level of correctness for our output was substantially higher compared to other sources of text created manually (88.4), templated (82.6) or generated by Inbox AI (91.8). A potential reason for this superior quality could be the extensive editorial-quality corpus our LLM is trained against and from which it derived its syntax and style knowledge. Users were quite satisfied with their experiences in general (4.6/5.0); they particularly liked being able to customize the tone of their messages and felt that they did not need to do much editing before sending. When satisfaction was examined within various categories of e-mail—formal business, complaint, etc.—formal business provided the highest average satisfaction (4.8), while complaints and escalations were rated the lowest, but still positive (4.3). Some users did mention that the phrasing generated by the model was sometimes lacking the appropriate assertiveness of a complaint e-mail.

F. Latency Analysis

It took an average of 2.8 seconds for a user to see their email draft appear after they clicked ‘Generate’ on our user interface (UI). This time can be broken down into three components; the time taken for a network to transmit data to our customer (approx. 120 msec), the time taken by our main server to construct and validate a prompt (approx. 80 msec), and the time taken for our LLM to process this information and produce an output (2,600 msec). The processing component accounts for approximately 93% of the total latency. Therefore, these numbers indicate that any optimization efforts in the future should be directed towards model-side enhancements such as using speculative decoding, quantization (i.e., compressing large models into smaller versions), or distillation (i.e., simplifying complex LLMs). Because the 2.8 second average is far below the 5 second threshold that has been identified through usability research to keep users engaged in interactive tasks, and since no participant expressed any concern regarding the speed of their response during the group debrief after the experiment, it is reasonable to assume that existing LLM tool users would find our tool to have about the same time to receive a response as they did with existing tools thus, the time to respond is not likely to be a barrier to user adoption. Overall our proposed system provides a very positive score based on what we call an aggregate sum of five metrics. Drafting is completed quickly, editing requirements are minimal, high grammar scores result from drafts, users express a high level of satisfaction, and there’s acceptable latency associated with receiving the drafted email. The data suggests that users will have a positive experience when using an LLM to create an email in a professional manner. The greatest level of impact on the email generations process occurs when using LLM to generate an email with a specific tone or style; in particular, an email that is considered ‘formal’ or ‘escalation.’

6. CONCLUSION

This study describes an email automation system that utilizes large language models to create a polished,

contextually appropriate email draft based on a brief set of user inputs. The use of layered architecture—presentation, processing, persistence, inference—allows for a modular, extensible, and independent system that does not rely on one technology stack to operate. The data from our research shows that using our system resulted in an average of 70% less time needed for writing an email draft, 15% or less effort required for editing emails, grammar scores of 95 or above out of 100 for emails generated using our system, and user satisfaction ratings of 4.5 or above on a 5-point scale. Therefore, these results support the assertion that LLM-generated email drafts provide a significant improvement over traditional methods of automating email writing and using platform-specific AI assistants for writing emails that require nuanced tone. The platform-independent design and modular structure of our system provide a strong basis for enterprise implementation and future research into the use of AI to help professionals communicate. Therefore, the substantial productivity gains produced by a 70% reduction in average draft time become greatly increased when this is applied across the entire volume of emails sent within a company due to minimal editing required for review. Furthermore, low editing effort reduces the time usually spent reviewing AI written content, which is one of the main issues affecting the usefulness of AI writing systems in practice. A limitation of this study is the sampling method used to gather participants for this study; despite the large geographical diversity in experience among participants, they were all sampled from the same location.

7. FUTURE WORK

The research has many avenues for extension. They include:

i. Multilingual Generation:

Supporting multilingual email writing across different nationalities and cultures would provide support for both multilingual LLM and translation-augmented pipelines in a global organization. Initial testing has shown that some of the newer multilingual models do well for high-resource languages, but further evaluation needs to be completed for lower-resource languages.

ii. User Memory and Personalization:

Supporting a user profile that permanently tracks a user's preferred style, phrases, and prior edits would allow the model to produce more customized draft content. Use of retrieval plus augmented generation could be used to condition the model based on prior users of the user's email drafts.

a. Enterprise Integration:

Building connectors to other ERP systems (calendars, CRMs, Project etc.) will allow the model to incorporate contextual information from outside workflows into drafts.

b. Sentiment-Aware Tone Adjustment:

While generating response drafts in real time, the model will analysis the tone of previous email exchanges to ensure that the

response is produced with an appropriate register/tone. This would be beneficial to customer service and/or escalation scenarios in which using the correct tone will influence the success of a situation.

c. On-Device Inference:

Employing distilled or quantized methods of generating responses within the email application as a means of allowing the user to receive responses faster and with less latency than through existing server based methods for producing email responses.

REFERENCES:

- [1] S. Radicati and J. Levenstein, "Email statistics report, 2023– 2027," The Radicati Group, Inc., Palo Alto, CA, 2023.
- [2] M. Chui, J. Manyika, J. Bughin, R. Dobbs, C. Roxburgh, H. Sarrazin, G. Sands, and M. Westergren, "The social economy: Unlocking value and productivity through social technologies," McKinsey Global Institute, Jul. 2012.[Online]. Available: <https://www.mckinsey.com/industries/technologymedia-andtelecommunications/our-insights/the-socialeconomy>
- [3] E. Reiter and R. Dale, "Building applied natural language generation systems," *Natural Language Engineering*, vol. 3, no. 1, pp. 57–87, 1997.
- [4] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, and I. Sutskever, "Language models are unsupervised multitask learners," *OpenAI Technical Report*, 2019.
- [5] T. Brown et al., "Language models are few-shot learners," in *Proc. NeurIPS*, vol. 33, pp. 1877–1901, 2020.
- [6] T. Mikolov, M. Karafiat, L. Burget, J. Cernocky, and S. Khudanpur, "Recurrent neural network based language model," in *Proc. INTERSPEECH*, pp. 1045–1048, 2010.
- [7] P. Domingos and M. Pazzani, "On the optimality of the simple Bayesian classifier under zero-one loss," *Machine Learning*, vol. 29, pp. 103–130, 1997.
- [8] M. X. Chen et al., "Gmail Smart Compose: Real-time assisted writing," in *Proc. ACM KDD*, pp. 2287–2295, 2019.
- [9] A. Vaswani et al., "Attention is all you need," in *Proc. NeurIPS*, pp. 5998–6008, 2017.
- [10] Microsoft Corp., "Microsoft 365 Copilot for Outlook," *Microsoft Learn*, 2024. [Online]. Available: <https://learn.microsoft.com/enus/copilot/microsoft365/microsoft-365-copilot-overview>. Accessed: Jan. 15, 2025.
- [11] Google LLC, "Gemini for Google Workspace," *Google Workspace Blog*, 2024. [Online]. Available: <https://workspace.google.com/blog/productannouncements/gemini-for-google-workspace>. Accessed: Jan. 15, 2025.
- [12] X. Zhang, J. Zhao, and Y. LeCun, "Character-level convolutional networks for text classification," in *Proc. NeurIPS*, pp. 649–657, 2015.
- [13] K. Singhal et al., "Large language models encode clinical knowledge," *Nature*, vol. 620, pp. 172–180, 2023.
- [14] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A systematic survey of prompt engineering in large language models: Techniques and applications," *arXiv preprint arXiv:2402.07927*, 2024.
- [15] S. Gehrmann, E. Clark, and T. Sellam, "Repairing the cracked foundation: A survey of obstacles in evaluation practices for generated text," *J. Artif. Intell. Res.*, vol. 77, pp. 103–166, 2023.